تقرير حول معالجة الملفات الضخمة
التي تتجاوز حجم الذاكرة

Big Data --- العمل التطبيقي الثاني (TP2)

ملخص

يعرض هذا التقرير محتوى دفتر Jupyter المسمّى Tp2_code.ipynb، وهو تجربة على بيئة Google Colab لمعالجة ملف CSV حجمه نحو 5.5 غيغابايت ويضم 148 مليون سطر، أي أكبر من الذاكرة التي يحتاجها pandas لقراءته دفعة واحدة. قورنت ثلاث طرق: القراءة على أجزاء بـ pandas، ومكتبة Dask، ونسخة Dask محسَّنة. أعطت الطرق الثلاث المتوسط نفسه (50.0012)، وكان أسرعها Dask المحسَّنة بزمن 89.9 ثانية، وأقلها استهلاكاً للذاكرة القراءة على أجزاء بـ pandas بذروة 185 ميغابايت.

1 مقدمة وهدف العمل

تظهر مشكلة الذاكرة في البيانات الضخمة حين يكون حجم الملف أكبر من الذاكرة العشوائية (RAM) المتاحة، فتفشل القراءة المعتادة بـ pandas. يقارن الدفتر بين حلّين: تقسيم الملف إلى أجزاء صغيرة تُعالَج واحداً بعد الآخر، واستخدام Dask التي تقسّم الملف إلى أجزاء (partitions) وتؤجّل الحساب حتى طلب النتيجة. المهمة المنفَّذة هي حساب متوسط العمود value وتوزيع العمود category، وفي النسخة المحسَّنة حساب القيمة العليا أيضاً.

2 بيئة التنفيذ

جدول 1: مواصفات بيئة Colab كما طبعها الدفتر

العنصر	القيمة
الذاكرة العشوائية الكلية	12.7 GB
عدد أنوية المعالج	2
القرص	108G منها 88G متاحة
المكتبات	dask[complete] وpsutil وpyarrow وgraphviz (إضافة إلى pandas وnumpy وmatplotlib)

وبما أن الجهاز بنواتين فقط، فمقدار التوازي الذي تستفيد منه Dask محدود، وهذا يفسّر أن الفرق في الزمن بين الطرق متواضع.

3 توليد البيانات

يولّد الدفتر ملفاً اصطناعياً بإلحاق أجزاء من مليون سطر في كل مرة، ويتوقف حين يبلغ الحجم 5.5 غيغابايت. أعمدة الملف خمسة:

جدول 2: أعمدة الملف المولَّد

العمود	النوع	طريقة التوليد
`id`	عدد صحيح	أرقام متسلسلة تبدأ من 60,000,000
`value`	عشري	قيمة عشوائية منتظمة بين 0 و100
`price`	عدد صحيح	عشوائي بين 1 و9999
`category`	نصي	أحد الحروف A, B, C, D, E
`text`	نصي	إحدى الكلمات alpha, beta, gamma, delta

يُكتب العنوان (header) في أول مرة فقط، ويُحسب الحجم بدالة مساعدة تُرجع صفراً إن لم يكن الملف موجوداً لتفادي خطأ FileNotFound. طبع الدفتر بعد التوليد: الحجم 5.51 GB والأسطر المكتوبة في هذه الجلسة 146,000,000.

4 لماذا تفشل القراءة المعتادة؟

4.1 تقدير الذاكرة المطلوبة

قُرئ أول مليون سطر وقيس استهلاكه بـ `memory_usage(deep=True)`، ثم ضُرب في 148 لتقدير الملف كاملاً:

جدول 3: تقدير الذاكرة اللازمة لقراءة الملف دفعة واحدة

القياس	القيمة
مليون سطر في الذاكرة	122 MB
الملف كاملاً (148 مليون سطر)	17.6 GB
الذاكرة المتاحة في Colab	12.7 GB
نسبة المطلوب إلى المتاح	نحو 1.39 مرة

أي إن الملف في الذاكرة يزيد على 17 غيغابايت، بينما ملفه على القرص 5.5 غيغابايت فقط. سبب هذا التضخم أن pandas تخزّن الأعمدة النصية ككائنات Python، وهي أثقل بكثير من حجمها في الملف.

4.2 اختبار الفشل

ضُبط حدّ للذاكرة الافتراضية للعملية (`RLIMIT_AS`) بقيمة الاستهلاك الحالي مضافاً إليه 3 غيغابايت، ثم حُاولت قراءة الملف كاملاً بـ `pd.read_csv`. كانت النتيجة `MemoryError`، وبقيت نسبة استخدام الذاكرة قبل التجربة وبعدها (11.7% ثم 11.9%) شبه ثابتة، لأن الفشل جاء من الحد المفروض على العملية قبل أن تمتلئ ذاكرة الجهاز.

5 الطرق الثلاث للمعالجة

5.1 الطريقة 1: pandas على أجزاء

تُقرأ بيانات الملف بحجم 100,000 سطر لكل جزء (chunksize). في كل جزء يُضاف مجموع value وعدّه إلى متغيرين تراكميين، ويُحدَّث توزيع category في قاموس، ثم يُسجَّل أعلى استهلاك للذاكرة. بعد آخر جزء يُحسب المتوسط من المجموع والعدّ. نتائج هذه الطريقة:

- عدد الأجزاء: 1,480، والأسطر: 148,000,000.
- المجموع: 7,400,178,615.63، والمتوسط: 50.0012.
- توزيع category: كل حرف نحو 29.6 مليون سطر (أي 20% لكل فئة).
- الزمن: 123.8 ثانية، وذروة الذاكرة: 185 ميغابايت.

5.2 الطريقة 2: Dask

قُرئ الملف بـ dd.read_csv بحجم جزء 128MB، فنتج 46 جزءاً. أمر حساب المتوسط (mean) لا ينفَّذ فوراً بل يعيد كائناً مؤجَّلاً من نوع Scalar، ولا يبدأ الحساب إلا عند استدعاء ()compute. استُعملت ProgressBar لعرض التقدم وResourceProfiler لقياس الذاكرة. النتائج: المتوسط 50.0012، والزمن 102.4 ثانية، وذروة الذاكرة 1787 ميغابايت.

5.3 الطريقة 3: Dask المحسَّنة

أُضيفت ثلاثة تحسينات عند القراءة:

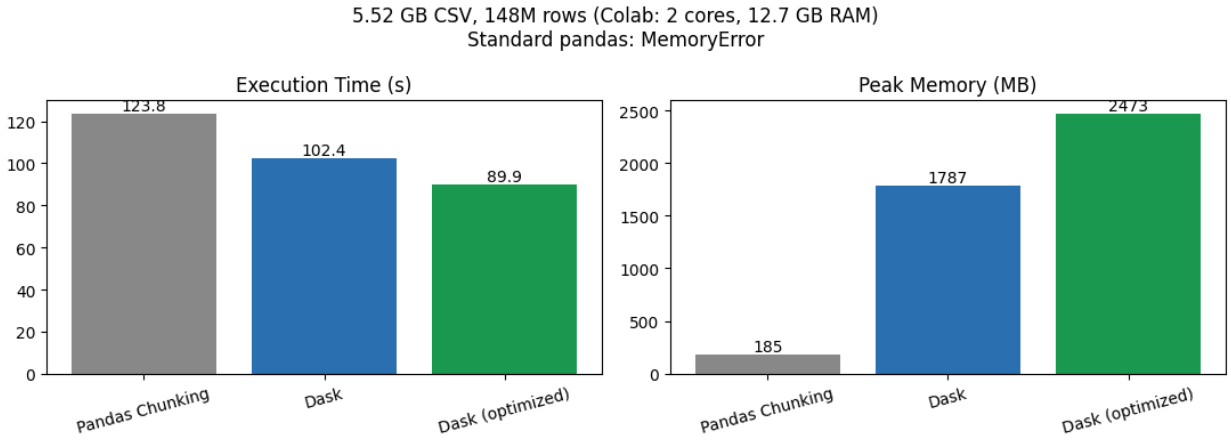
1. قراءة العمودين المطلوبين فقط (usecols): value وcategory.
2. تصغير الأنواع: value إلى float32 وcategory إلى نوع category.
3. حساب ثلاث نتائج دفعة واحدة بـ dask.compute: المتوسط والقيمة العليا وتوزيع الفئات، فيُمرَّ على الملف مرة واحدة.

النتائج: المتوسط 50.0012، والقيمة العليا 100.00، والزمن 89.9 ثانية، وذروة الذاكرة 2473 ميغابايت. ظهرت في توزيع الفئات صفوف باسم _CATEGORIES____UNKNOWN بعدّ صفر، لأن Dask لا تعرف قائمة الفئات مسبقاً، فاستُبعدت بحذف الصفوف التي عدّها صفر (counts، أي أكبر من صفر فقط)، وبقيت الفئات الخمس بأعدادها الصحيحة.

6 المقارنة والنتائج

جدول 4: مقارنة الطرق الثلاث (ملف 5.52 GB، 148 مليون سطر)

الطريقة	الزمن (ثانية)	ذروة الذاكرة (ميغابايت)	المتوسط
Pandas Chunking	123.8	185	50.0012
Dask	102.4	1787	50.0012
Dask (optimized)	89.9	2473	50.0012



شكل 1: الرسم الذي أنتجه الدفتر: زمن التنفيذ (يمين) وذروة الذاكرة (يسار)

بحساب الفروق بين الطرق من الأرقام نفسها:

جدول 5: الفروق النسبية (محسوبة من الجدول السابق)

الطريقة	التوفير في الزمن مقابل **Pandas**	الذاكرة مقابل **Pandas**	الأسطر في الثانية
Pandas Chunking	---	1.0x	1.20M
Dask	17.3%	9.7x	1.45M
Dask (optimized)	27.4%	13.4x	1.65M

الاستنتاجات:

1. القراءة المعتادة بـ pandas غير ممكنة لهذا الملف (MemoryError)، والطرق الثلاث كلها تتجاوز المشكلة.
2. الطرق الثلاث أعطت النتيجة نفسها (المتوسط 50.0012، وهو قريب جداً من القيمة النظرية 50 لتوزيع منتظم بين 0 و100)، فاختلافها في الأداء لا في الصحة.
3. **الأسرع:** Dask المحسَّنة، وهي أسرع من Dask العادية بنحو 12%.

4. **الأقل استهلاكاً للذاكرة:** pandas على أجزاء، بنحو 1.4% فقط من ذاكرة الجهاز، وهي أقل من Dask بنحو عشرة أضعاف.

5. هناك مقايضة واضحة: Dask تربح قليلاً في الزمن وتدفع أضعافاً في الذاكرة على جهاز بنواتين.

7 ملاحظات وتوصيات

1. **اختلاف اسم الملف:** التوليد يكتب في `big_files.csv` (بحرف s)، بينما القراءة تتم من `big_file.csv`. يعمل الدفتر إن كان الاسمان قد وُحِّدا يدوياً، وإلا فالأفضل استخدام متغير `path` واحد في كل الخلايا.

2. **عدد الأسطر:** طُبع 146,000,000 عند التوليد، بينما استعمل باقي الدفتر 148,000,000 (وهو ما تؤكده النتائج). والفرق مليونا سطر ربما بقيا من تشغيل سابق، لأن الكتابة بوضع الإلحاق (`mode="a"`) تبدأ من الملف الموجود، فالأفضل حذف الملف قبل التوليد أو حساب الأسطر من الملف نفسه.

3. **أرقام مكتوبة يدوياً:** خلية الرسم تحتوي الأزمنة والذاكرة وحجم الملف (5.52 GB) مكتوبة بيد المستخدم، والحجم المطبوع عند التوليد 5.51. الأفضل قراءتها من ملفات JSON التي حفظها الدفتر (`results_chunk.json` و`results_dask.json`)، وحفظ نتيجة الطريقة الثالثة في ملف مماثل.

4. **قياس الذاكرة غير متطابق:** pandas تقيس ذاكرة العملية (`rss`) بينما يقيس `ResourceProfiler` ذاكرة Dask بطريقة مختلفة، فالمقارنة تقريبية.

5. **ذاكرة النسخة المحسَّنة أعلى:** المتوقع أن تنخفض الذاكرة بقراءة عمودين فقط وبأنواع أصغر، لكنها ارتفعت من 1787 إلى 2473 ميغابايت. قد يكون السبب حساب ثلاث نتائج معاً أو معالجة أجزاء متوازية، لكن الدفتر لا يثبت السبب. وتُجرى التجربة مرة واحدة لكل طريقة، فيُستحسن تكرارها عدة مرات وأخذ المتوسط قبل الجزم، أو تقليل `blocksize`.

6. **فئات غير معروفة:** لتفادي صفوف `_CATEGORIES____UNKNOWN_` يمكن تعريف الفئات مسبقاً بالنوع `CategoricalDtype` أو استدعاء `.cat.as_known()`.

7. **مكتبات غير مستعملة:** ثُبِّتَت `graphviz` و`pyarrow` ولا يُستعملان (لا رسم لمخطط المهام بـ `visualize`، ولا ملفات Parquet)، وخلية التثبيت مكررة مرتين. تحويل الملف إلى Parquet بـ `pyarrow` يُتوقَّع أن يسرّع القراءة كثيراً لأنه ملف عمودي مضغوط.

8. **الحد على الذاكرة الافتراضية:** `RLIMIT_AS` يقيّد العملية لا الجهاز، فهو يُظهر الخطأ بسلاسة لكنه لا يحاكي امتلاء الذاكرة الحقيقية.

8 الخاتمة

أثبت العمل أن ملفاً حجمه 5.5 غيغابايت (نحو 17.6 غيغابايت في الذاكرة) لا يمكن قراءته دفعة واحدة بـ pandas على جهاز ذاكرته 12.7 غيغابايت، وأن المعالجة على أجزاء أو بـ Dask تحل المشكلة وتعطي النتائج الصحيحة نفسها. اختيار الطريقة يعتمد على القيد الأهم: **الذاكرة المحدودة** تناسبها pandas على أجزاء، و**السرعة** مع توفر ذاكرة أكبر وأنوية أكثر تناسبها Dask، وتزداد فائدتها مع التحسينات وعدد الأنوية.